

Python (3.13.1) ▼

```
1 words = input().split()
2
3 for w in words:
4     if w == w[::-1]:
5         print(w)
6         break
7 else:
8     print("")
```

INPUT

OUTPUT

ERROR

1 abc car ada racecar cool

INPUT	OUTPUT	ERROR
1	ada	
2		

Python (3.13.1) ▼

```
1  nums1 = list(map(int, input().split()))
2  nums2 = list(map(int, input().split()))
3
4  a1 = sum(x in nums2 for x in nums1)
5  a2 = sum(x in nums1 for x in nums2)
6
7  print([a1, a2])
```

INPUT

OUTPUT

ERROR

1 1 2 3 4

2 3 4 5 6

INPUT	OUTPUT	ERROR
1	[2, 2]	
2		

Python (3.13.1) ▾

```
1  nums = list(map(int, input("Enter numbers: ").split()))
2
3  ans = 0
4  for i in range(len(nums)):
5      s = set()
6      for j in range(i, len(nums)):
7          s.add(nums[j])
8          ans += len(s) ** 2
9
10 print(ans)
```

INPUT	OUTPUT	ERROR
-------	--------	-------

1	1 2 1	
---	-------	--

INPUT	OUTPUT	ERROR
1	Enter numbers: 15	
2		

Python (3.13.1) ▾

```
1  nums = list(map(int, input("Enter array: ").split()))
2  k = int(input("Enter k: "))
3
4  count = 0
5  for i in range(len(nums)):
6      for j in range(i + 1, len(nums)):
7          if nums[i] == nums[j] and (i * j) % k == 0:
8              count += 1
9
10 print(count)
```

INPUT | OUTPUT | ERROR

1 Enter array: Enter k: 4

2

Python (3.13.1) ▼

```
1 arr = list(map(int, input("Enter numbers: ").split()))  
2 print(max(arr))
```

INPUT

OUTPUT

ERROR

1 1 2 3 4 5

INPUT	OUTPUT	ERROR
1	Enter numbers: 5	
2		

```
1  arr = list(map(int, input("Enter numbers: ").split()))
2
3  if len(arr) == 0:
4      print("List is empty")
5  else:
6      arr.sort()
7      print("Sorted List:", arr)
8      print("Maximum Element:", arr[-1])
```

UT	OUTPUT	ERROR
----	--------	-------

5 2 9 1 7

INPUT	OUTPUT	ERROR
1	Enter numbers: Sorted List: [1, 2, 5, 7, 9]	
2	Maximum Element: 9	
3		

Python (3.13.1) ▼

```
1  arr = list(map(int, input("Enter numbers: ").split()))
2
3  unique = []
4  for i in arr:
5      if i not in unique:
6          unique.append(i)
7
8  print("Unique Elements:", unique)
```

INPUT	OUTPUT	ERROR
-------	--------	-------

1	3 7 3 5 2 5 9 2	
---	-----------------	--

INPUT	OUTPUT	ERROR
1	Enter numbers: Unique Elements: [3, 7, 5, 2, 9]	
2		

Python (3.13.1) ▼

```
1  arr = list(map(int, input("Enter numbers: ").split()))
2
3  n = len(arr)
4
5  for i in range(n):
6      for j in range(0, n-i-1):
7          if arr[j] > arr[j+1]:
8              arr[j], arr[j+1] = arr[j+1], arr[j]
9
10 print("Sorted Array:", arr)
```

INPUT

OUTPUT

ERROR

1 5 1 4 2 8

INPUT	OUTPUT	ERROR
1	Enter numbers: Sorted Array: [1, 2, 4, 5, 8]	
2		

Python (3.13.1) ▾

```
1  arr = sorted(list(map(int, input("Enter array: ").split()))))
2  key = int(input("Enter key: "))
3
4  low = 0
5  high = len(arr) - 1
6
7  while low <= high:
8      mid = (low + high) // 2
9      if arr[mid] == key:
10         print("Element found at position", mid + 1)
11         break
12     elif arr[mid] < key:
```

INPUT

OUTPUT

ERROR

1 3 4 6 -9 10 8 9 30

2 10

INPUT	OUTPUT	ERROR
1	Enter array: Enter key: Element found at position 7	
2		

Python (3.13.1) ▾

```
1  def merge_sort(arr):
2      if len(arr) <= 1:
3          return arr
4
5      mid = len(arr) // 2
6      left = merge_sort(arr[:mid])
7      right = merge_sort(arr[mid:])
8
9      result = []
10     i = j = 0
11
12     while i < len(left) and j < len(right):
```

INPUT

OUTPUT

ERROR

1 Enter numbers: Sorted Array: [1, 2, 5, 7, 9]

2

INPUT	OUTPUT	ERROR
1	Enter numbers: Sorted Array: [1, 2, 5, 7, 9]	
2		

```
1  from functools import lru_cache
2
3  m = int(input("Rows: "))
4  n = int(input("Columns: "))
5  N = int(input("Steps: "))
6  i = int(input("Start Row: "))
7  j = int(input("Start Column: "))
8
9  @lru_cache(None)
10 def dfs(x, y, steps):
11     if x < 0 or x >= m or y < 0 or y >= n:
12         return 1
```

INPUT

OUTPUT

ERROR

1 2

2 2

3 2

INPUT	OUTPUT	ERROR
-------	--------	-------

1	Rows: Columns: Steps: Start Row: Start Column: 6	
---	--	--

2		
---	--	--

Python (3.13.1) ▼

```
1  nums = list(map(int, input("Enter money in houses: ").split()))
2
3  def rob(arr):
4      prev = curr = 0
5      for money in arr:
6          prev, curr = curr, max(curr, prev + money)
7      return curr
8
9  if len(nums) == 1:
10     print(nums[0])
11 else:
12     print(max(rob(nums[:-1]), rob(nums[1:])))
```

INPUT

OUTPUT

ERROR

1 2 3 2

INPUT	OUTPUT	ERROR
1	Enter money in houses: 3	
2		

Python (3.13.1) ▾

```
1  n = int(input("Enter number of steps: "))
2
3  a, b = 1, 1
4  for i in range(n):
5      a, b = b, a + b
6
7  print(a)
```

INPUT

OUTPUT

ERROR

1

4

INPUT	OUTPUT	ERROR
1	Enter number of steps: 5	
2		

Python (3.13.1) ▼

```
1  import math
2
3  m = int(input("Enter rows: "))
4  n = int(input("Enter columns: "))
5
6  print(math.comb(m + n - 2, m - 1))
```

INPUT

OUTPUT

ERROR

1 7

2 3

INPUT	OUTPUT	ERROR
1	Enter rows: Enter columns: 28	
2		

```
1  s = input("Enter string: ")
2
3  result = []
4  i = 0
5
6  while i < len(s):
7      j = i
8      while j < len(s) and s[j] == s[i]:
9          j += 1
10
11     if j - i >= 3:
12         result.append([i, j - 1])
```

INPUT

OUTPUT

ERROR

	INPUT	OUTPUT	ERROR
1	abbxxxxzy		

INPUT	OUTPUT	ERROR
1	Enter string: [[3, 6]]	
2		

Python (3.13.1) ▼

```
1  def gameOfLife(board):
2      rows = len(board)
3      cols = len(board[0])
4
5      directions = [(-1,-1),(-1,0),(-1,1),
6                    |      |      |      |      (0,-1),          (0,1),
7                    |      |      |      |      (1,-1),(1,0),(1,1)]
8
9      result = [[0]*cols for _ in range(rows)]
10
11     for i in range(rows):
12         for j in range(cols):
```


Python (3.13.1) ▼

```
29
30 board = [
31     [0,1,0],
32     [0,0,1],
33     [1,1,1],
34     [0,0,0]
35 ]
36
37 ans = gameOfLife(board)
38
39 for row in ans:
40     print(row)
```

INPUT

OUTPUT

ERROR

1	board = [[0,1,0],
2	[0,0,1],
3	[1,1,1],

TV

INPUT	OUTPUT	ERROR
1	[0, 0, 0]	
2	[1, 0, 1]	
3	[0, 1, 1]	

Python (3.13.1) ▼

```
1 poured = int(input("Enter poured cups: "))
2 query_row = int(input("Enter query row: "))
3 query_glass = int(input("Enter query glass: "))
4
5 dp = [[0.0] * 101 for _ in range(101)]
6 dp[0][0] = poured
7
8 for i in range(query_row + 1):
9     for j in range(i + 1):
10         extra = (dp[i][j] - 1.0) / 2.0
11         if extra > 0:
12             dp[i + 1][j] += extra
```

INPUT

OUTPUT

ERROR

1 2

2 1

3 1

INPUT	OUTPUT	ERROR
-------	--------	-------

1	Enter poured cups: Enter query row: Enter query glass: Champagne in glass: 0.5	
---	--	--

2		
---	--	--